RetailPulse AI

Team pitch and responsibility split

Five speakers | Target presentation: 5-6 minutes, followed by a demo

A retail platform connecting shopping, store operations and policy assistance in one experience.

Team responsibilities

Member	Ownership	Demo focus
Sukh | Backend	Architecture, database, authentication and permissions	Customer, manager and admin access
Vishant | Backend	Catalogue, cart, checkout, orders and stock consistency	Store stock and order placement
Yug | Backend	Support, manager operations, replenishment and audit records	Ticket routing and controlled stock updates
Dilpreet | Frontend	Shopping interface, filters, cart and customer experience	Browsing through checkout
Suhsnt | Frontend + RAG	Chat interface, policy retrieval and knowledge panel integration	Policy answers with citations

This is a proposed presentation split based on the project. Adjust ownership statements to match each member's actual contributions.

Demo sequence

Role access > Shopping and checkout > Support and operations > Policy question with citations

MVP scope: payments are simulated. Demand forecasting and fully autonomous operations are future work.

1. Sukh | Backend architecture and security

Approximately 70 seconds

Good morning, respected panelists. We are presenting RetailPulse AI, a retail platform that brings shopping, store operations and policy assistance into one connected experience.

In retail, customers need accurate product availability and clear answers, while store teams need reliable orders, inventory and support workflows. Our project connects these needs through a shared backend.

I will explain our backend architecture and security. We use Node.js and Express for the APIs, with Prisma and Neon PostgreSQL for data storage. We separate routes, controllers and business services so each part has a clear responsibility.

Our system supports three access roles: customer, store manager and administrator. Customers access their own orders, managers handle their assigned locations, and administrators manage operations across locations. Membership tiers separately determine benefits and applicable policies.

This gives the platform a controlled foundation: every request is checked before it can access or change business data.

Vishant will now explain how we keep shopping and inventory consistent.

2. Vishant | Shopping, checkout and inventory

Approximately 60 seconds

My part of the backend covers the shopping journey: product discovery, cart management, checkout and orders.

Our catalogue connects each product to inventory at individual locations. When a customer selects a store, they see that store's assortment. Product filters help them narrow results by price, brand, rating and category-specific specifications.

The most important check happens during checkout. Adding an item to the cart does not reserve it, so the backend checks stock again when the customer places the order. Order creation and stock updates run within a database transaction.

We also handle repeated checkout requests. If a customer retries the same checkout, the backend returns the existing order instead of creating a duplicate.

The platform supports pickup and delivery, with order status rules enforced by the backend. Payments are simulated in this MVP.

These controls make the shopping flow dependable. Yug will explain how store teams manage the work after an order is placed.

3. Yug | Support and store operations

Approximately 60 seconds

My backend focus is support and store operations.

When a customer raises a ticket about an order, the system checks that the order belongs to them and routes the ticket to its actual fulfillment location. Delivery-order tickets go to the warehouse team, while store-order tickets go to the relevant store manager.

Managers can reply, escalate and resolve tickets within their assigned locations. Customers can see the replies with the author's role and timestamp. A ticket requires a staff response before it can be resolved.

We also support inventory replenishment through a controlled workflow. A replenishment proposal requires staff approval, and stock changes only after receipt is explicitly recorded. Audit records help trace these operational actions.

The value is clear ownership: customers know where to get help, and staff work with the orders and inventory they are responsible for.

Dilpreet will now show how these backend capabilities appear in the customer interface.

4. Dilpreet | Frontend experience

Approximately 45 seconds

I will demonstrate the customer experience: selecting a store, browsing and filtering products, adding items to the cart, and choosing pickup or delivery. The interface also connects customers to their orders and support conversations. Our aim is to make these connected workflows easy to use from one place.

Suhsnt will now demonstrate how customers find answers to retail policy questions.

5. Suhsnt | Frontend and RAG

Approximately 60 seconds

Our assistant uses retrieval-augmented generation, or RAG, to answer questions using uploaded retail policies. Documents are split into passages, converted into embeddings and stored using PostgreSQL with pgvector. Before retrieval, the backend checks which policies apply to the user's role, membership tier and policy dates. The assistant then answers using the retrieved passages and provides source citations. When there is insufficient supporting evidence, it directs the user to support. I will demonstrate this through the chat interface and the administrator's knowledge panel.

Closing | Sukh

RetailPulse AI connects a usable shopping experience, controlled retail operations and policy answers backed by sources. Thank you - we are ready to demonstrate the complete journey and take your questions.